2023 IEEE 8th International Conference On Software Engineering and Computer Systems (ICSECS) | 979-8-3503-1093-1/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICSECS58457.2023.10256409

An Overview Analysis of Authentication Mechanism in Microservices-based Software Architecture: A Discussion Paper

Hoong Jack, Chang
Department of Software Engineering And Technology
Tunku Abdul Rahman University of Management and Technology
Kuala Lumpur, Malaysia
changhj@tarc.edu.my
https://orcid.org/0009-0003-0614-7521

Kwee Teck, See
Department of Information System And Security
Tunku Abdul Rahman University of Management and Technology
Kuala Lumpur, Malaysia
seekt@tarc.edu.my

Tong Ming, Lim
Department of Centre For Business Incubation And Entrepreneurial Venture
Tunku Abdul Rahman University of Management and Technology
Kuala Lumpur, Malaysia
limtm@tarc.edu.my

Ying Hong, Ding
Department of Software Engineering And Technology
Tunku Abdul Rahman University of Management and Technology Kuala Lumpur, Malaysia
dingyh@tarc.edu.my

***Abstract*—Microservices-based software architecture promotes scalability and flexibility by breaking down a software application into smaller modules and making it more independent and loosely coupled services compared to monolith systems. However, securing microservices in a distributed nature has become one of the challenges. Authentication is one of the most critical components that should be focused in the microservices security measures. It helps to identify that only authenticated personnel and services can access sensitive information and secure the trust between microservices. This discussion paper aims to provide an overview analysis and extensive understanding on the authentication mechanism in microservices-based software architecture. In this study, we explore different authentication mechanisms including Mutual Transport Layer Security (mTLS), Token-based authentication and API Gateway authentication. This study examines the strengths and limitations of different authentication mechanisms in microservices-based software architecture. It also emphasizes the importance of authentication and the need for having a well-designed authentication mechanism to ensure the integrity and security of microservices-based software architecture is crucial.**

Keywords*—*microservices, architecture, authentication mechanism, security

I. Introduction

With the high-speed growth of Internet Technology, microservices-based software architecture has gained a certain popularity in building distributed software systems. Microservices can be known as small-scale or portable services which are deployed and run in each virtual machine (VMs) or independent containers [1] to reduce the coupling between services modules and beneficial to the management of huge application systems [11]. Services that run independently in different containers could bring several advantages to the microservices over the traditional monolith architecture. In the study of [12], researchers mentioned that the software industry is looking to build systems that can be managed to reach a higher level of maintainability and scalability. Therefore, separation of services provides independent deployment, updates and also management on each isolated services which allow continuous support and upgrade independently for each business function [2]. However, with the increasing use of the microservice architecture, secure communication between interconnected services has set off a critical concern. The study of [3] mentioned that in the past few years, there is a rapid adoption of microservices but less attention on security. Researcher in [12] mentioned that the adoption of independent user authentication is the key for microservices-based business systems. Therefore, authentication mechanisms play an essential role in securing the microservices-based software architecture by authenticating the identities of the parties and establishing trust communication between each service to prevent illegal access to the services. Because of the distributed nature in microservice architecture, each service has an entry point that needs to be authenticated. Recent studies have reported that security is a crucial challenge that requires attention because services are required to create a trust relationship when communicating with each other [8]. Therefore, a more complex or suitable authentication mechanism is needed to cater the authentication at each entry point of services.

This discussion paper aims to provide an overview anal-

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:57:20 UTC from IEEE Xplore. Restrictions apply.

ysis on the authentication mechanism that apply in the microservices-based software architecture. The paper explores a few types of authentication mechanisms that are commonly used to implement in microservices-based software architecture and discusses their strengths and limitations. This discussion paper mainly focuses on the use of API Gateway, Token-based authentication and Mutual Transport Layer Security (mTLS) authentication mechanism in the microservices-based architecture. Next, it delves into the understanding of microservice security authentication and the different variance of existing authentication mechanisms available.

Overall, the paper aims to provide a comprehensive analysis on the authentication mechanism for securing microservices when selecting and implementing authentication mechanisms in the microservices-based software system.

II. Literature Reviews

Much of the literature on microservices-based software architecture is extensive and focuses particularly on the authentication and authorization mechanism in the distributed nature. With the raising of microservices-based architecture, research indicates that security is one of the greatest challenges in a real-world distributed system, therefore the need to address the security concern in microservice-architecture is critical [3]. In the [3] research paper, researchers developed a small prototype framework to establish the trust and secure the communication using the baseline security, Mutual Transport Layer Security (mTLS) mechanism, token and the combinations of mTLS+token in the testing. In the testing result, it has an additional performance cost to secure the microservices communication due to higher overhead in the communication itself during the process. The performance of the authentication should be focused in the microservices process. Furthermore, mutual transport layer security, API Gateway and token-based authentication mechanism are discussed in [4] research paper to achieve service-to-service authentication in microservices-based architecture. Based on the researchers review, the study has identified drawbacks in mTLS authentication including the complexity of implementation for mTLS and required in-house developers to perform maintenance and fix vulnerabilities due to mTLS might not rely on any third party authentication server. Secondly, a single-point-of decision in API gateway has violated the ”defense in depth” principle which is likely to bring down the whole system if the API gateway failed to service. The authors identified best practices in authentication and authorization patterns and reviewed the advantages and disadvantages that could be used as the decision for applying security architecture during the development phase. The authors also reviewed that the use of Mutual Authentication Transport Security (mTLS) is still not widely introduced as a primary security mechanism in service-to-service communication [4] compared to API Gateway and Token-based authentication in microservices due to some drawbacks. Moreover, the paper [9] provides an understanding on the cloud-edge integrated security system with a set of secure access solutions for business organizations based on the API secure access gateway authentication in the cloud-edge with related technology such as Nginx and performed performance analysis to test on the overall reaction time with different authentication approaches. The paper also discussed the advantages of the Zero Trust model in improving security in a cloud-edge integration. Besides that, the greater part of the paper on [1] proposed particularly on the mutual authentication mechanism benchmark based on top of the standard TLS protocol. The study examines the performance and channel latency benchmarking in many different attested communication mechanism scenarios and achieved a modest performance in some of the attested mechanisms. The paper also discusses the need for strong authentication between microservices in large systems composed of many rapidly-deployed microservices that cross networks and span trust domains. In [5], the study proposed the use of certificates in securing IoT microservices provides a clear understanding on the certificate based methodology for authenticating services and addressing challenges related to authentication and validation of security mechanisms on distributed entities. The authors also discussed autonomous certificate management for the certificate renewal process in the IoT microservices system which is worth studying and useful to the current research study. Previous studies of [6] provided reviews of authentication and authorization in microservices architecture. The authors compared traditional security measures with existing technology and discussed a few sets of authentication and authorization approaches for microservices such as distributed session, API gateway, Single sign-on (SSO), token-based authentication(JWT) and JWT+API Gateway. The authors also summarized the advantages and disadvantages of each method. Overall, these studies have attempted to provide a summary of the literature relating to the authentication mechanism in these areas such as performances, advantages and disadvantages of each security mechanism. In view of all that has been mentioned so far, each of these security mechanisms carried out its own strengths and limitations. API Gateway and Token-based authentication mechanism are mostly serving as a centralized concept authentication mechanism when API gateway only provided one single entry point to the services and token based authentication rely on the third party issuer to issue a token. However, centralized authentication mechanisms have served as the foundation in the past, but it is clear that decentralized authentication presents its adaptability and capability in terms of its strengths and limitations in digital security and user access in the current dynamic digital landscape.

III. Methodology

The nature of user authentication is to provide verification of user identity who performed requests to the microservices to access information. Since microservices are formed of multiple independent services, the authentication mechanism plays a critical role to secure and establish the communication between all the microservices. This section discusses API gateway authentication, token-based authentication and mutual transport layer security authentication in terms of definitions,

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:57:20 UTC from IEEE Xplore. Restrictions apply.

strengths and limitations in order to understand the essential for enabling secure communication in a distributed nature.

A. API Gateway Authentication

API gateway is a common centralized authentication mechanism where it handles all the incoming requests and provides an unified entry point for clients that interact with these microservices [6]. It also provides features including traffic management, load balancing control, caching, permission granting, etc.

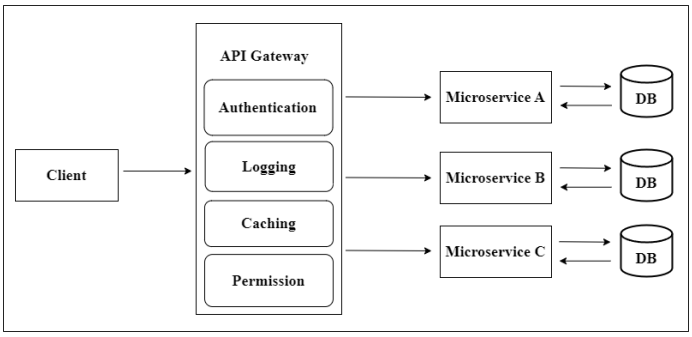

Fig. 1: An Overview in API Gateway Authentication [14]

As shown in the figure 1, API gateway authentication only provides a single and unified entry point to authenticate clients. The benefit of using API Gateway Authentication mechanism is it provides a centralized authentication logic to identify user incoming requests that ensures the communication between clients and microservices is secure. Monitoring and logging is a collection of traffic statistics and to manage the details about who accessed the specific APIs in the services. It helps to monitor each incoming request from the external and provide tracking for error and exceptions. Additional security features including TLS encryption and request filtering are introduced to secure the communication between client and microservices and validate incoming requests with predefined rules.

However, it also contains some limitations including performance overhead, security concern and the single point of failure. [2] concluded that the difficulties of implementing an API gateway fall under its ability to manage high volume requests and performance requirements. Therefore, an additional layer of security and centralized single entry point are possible to impact the performance because the API gateway needs to authenticate every single incoming request from the clients which leads to the increasing of latency. Next, the point of centralized authentication has brought up the security concern. [12] think that the use of gateway authentication consists of a huge security risk due to the services behind the API gateway being completely open and the the services behind the gateway will be fully compromised once the attacker crosses the gateway. Therefore, researchers from [4] mentioned that a single point of decision may violate the "defense in depth" principle due to some reasons such as dependence on third party providers and single point of failure. Moreover, by applying the API gateway authentication mechanism, all the microservices and clients depend on the API gateway to do the authentication process before communicating with each others. If the API gateway is down and this will lead to all the incoming request from the outside to the microservices fail because API gateway is the only entry point to the microservices.

B. Token-based Authentication

Token-based authentication mechanisms generate unique tokens to authenticate clients and microservices. The token generated from the token-based authentication server will be assigned to the client after successfully verifying the identity of the client mentioned by [7]. As shown in the figure2, the token-based authentication mechanism flow pattern running in the microservices-based architecture. We can see that when a client performs a login request to a microservice, the request information will be sent to the token-based authentication server to verify the user's identity and once the client is verified, the token will be assigned from the authentication server to the client. Once the client receives the token, the token is included in the header every time when the client performs a request to the microservices. The microservice will validate the token by checking its signature and verify the claims that include. The token usually contains information about the user information including user ID, email address, roles and the permission to access the resources in microservices.

The significant advantages of using token-based authentication in the distributed nature is the security of the token. Token needs to be configured to have a shorter lifespan and it should be signed and encrypted to ensure the information in the token cannot be easily intercepted. A signed and encrypted token plays a crucial role in preventing session hijacking and man-in-the-middle attacks. Next, token-based authentication provides flexibility because it provides integration with other third-party authentication providers and it also accepts different types of tokens including JSON Web Token (JWTs) and OAuth access token. Token-based authentication and token brings improvement in authentication experience by reducing the need for clients to enter the credentials repeatedly. Clients are able to maintain their access to the resources until the token expires.

However, this token-based authentication also has a number of drawbacks including slow performance, security considerations and single-point of failure. Slow performance is a significant concern in token-based authentication when it requires an additional network process to authenticate the clients from the incoming request. This performance delay can be noticeable in a higher volume traffic application which leads to slower response times and has a negative impact on the overall system performance. Secondly, security considerations are crucial when applying this authentication mechanism. [1] has mentioned that the private key of the token must be properly managed and secured to prevent interception by attackers. Moreover, if the private key is compromised, it may lead to compromised services, data breaches and unauthorized access to the resources by the attacker. Furthermore, token-based

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:57:20 UTC from IEEE Xplore. Restrictions apply.

authentication suffers from single-point of failure when the authentication server fails to serve. This limitation highlights that services that rely on a single third party authentication server may not be ideal towards highly distributed microservices applications.

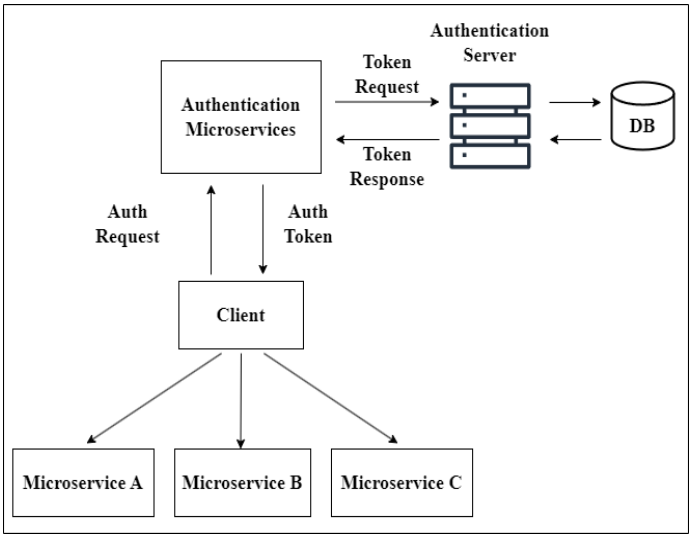

Fig. 2: An Overview in Token Based Authentication [15]

C. Mutual Transport Layer Security(mTLS)

Mutual Transport Layer Security (mTLS) is an authentication mechanism that provides secure communication across microservices in the distributed system to achieve confidentiality and integrity of the exchanged data [4]. A number of studies have shown that the mTLS mechanism is based on the foundation of Transport Layer Security (TLS) protocol which is a widely adopted security protocol that is designed to facilitate the privacy and data security of communications over the internet. With mTLS, each microservices obtained a digital certificate from the Certificate Authority (CA) to verify and establish a secure communication with other microservices in the distributed application. Figure 3 provides an overview of the mTLS authentication flow in the distributed system. Firstly, the client performs a request to the authentication server. Next, the authentication server performs verification on the client identity and generates a certificate to the client to establish the connection with the microservices. Moreover, the digital certificates are always required whenever the microservices try to establish a communication with each other to access the resource API. Additionally, mTLS authentication provides end-to-end encryption during the process of data exchange and data transmitting between microservices at the transport layer which adds an extra layer of security protection.

The main strength of mTLS authentication mechanism is it uses a certificate-based approach that is widely accepted as a secure measure of authentication. [7] Researcher claims that the two-sided certificate is appropriate in many different scenarios under the decentralized concept. The benefit of this mutual authentication is that it does not require a centralized authentication server or a token issuer process compared to other authentication mechanisms. This means that mTLS has the ability to work in a distributed environment with multiple microservices and ensure secure communication without the need of relying on a centralized authentication server. Additionally, mTLS provides strong end-to-end encrypted authentication based on the foundation of TLS protocol in the distributed nature and it supports message integrity and confidentiality during the data exchange process between microservices.

Unfortunately, the mTLS mechanism has certain limitations that need to be taken into consideration in terms of performance and complexity of its implementation and configuration. Firstly, mTLS may have potential performance impacts on the response time during the handshakes process. A small prototype carried out in the study [1] concluded that there is an additional performance cost in the mTLS to secure the communication due to the extra overhead from the transmission itself. As shown in figure 4, the handshakes process in mTLS shows the client and server exchanging certificates to authenticate each other to establish a secure connection. The process could be time-consuming when client and server need to perform multiple round trips to negotiate the encryption protocol and exchange the certificates. Also, if a certain amount of microservices are involved in a transaction process, each microservices needs to perform mTLS handshake separately which can increase significant latency and traffic level due to increasing number of communication between services [4]. Furthermore, complexity of mTLS implementation and configuration including the setup of public key infrastructure (PKI), generation of public and private key pairs, generation of certificate and the certificate authorities (CA). These are the core components that involve configuring the client and server to receive appropriate certificates and keys for authentication. The use of mTLS authentication mechanisms can be challenging to manage in a large and complex microservices-based system. Last but not least, mTLS faces difficulty in trust store management, which manages the trusted root certificates that are used to authenticate the genuineness of client and server certificates in each of the microservices. These trust stores can become a bottleneck in a large-scale microservices-based system and any misconfiguration may result in authentication failure.

IV. Result and Discussion

To review the strengths and limitations between each authentication mechanism that we discussed, a table is generated as shown in Table 1. Table 1 provides an overview of the strengths and limitations among all three authentication mechanisms discussed. It appears that all the mechanisms have their own strengths and limitations implementing the microservices-based architecture.

One of the aims of this study was to determine the strengths and limitations of authentication mechanism works in microservices-based architecture and provide understanding on each mechanism. It was found that API gateway mechanism

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:57:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Strengths and Limitations in Authentication Mechanisms

Authentication Mechanism	Strengths	Limitations
API Gateway	1. Centralized Authentication and Authorization 2. Provides monitoring and logging 3. Additional security features implementations	1. Performance Overhead 2. Security Concerns 3. Single Point of Failure 4. Reduce Flexibility
Mutual Transport Layer Security (mTLS)	1. Decentralized Authentication Mechanism 2. Provides strong end-to-end encrypted authentication 3. Support message integrity and confidentiality	1. Performance Impact 2. Complexity in implementation and configuration 3. Maintenance of certificates and keys
Token-based Authentication	1. Flexibility on integrations 2. Improvement on authentication experience 3. Security configuration on tokens	1. Single Point of Failure 2. Performance Overhead 3. Security Considerations

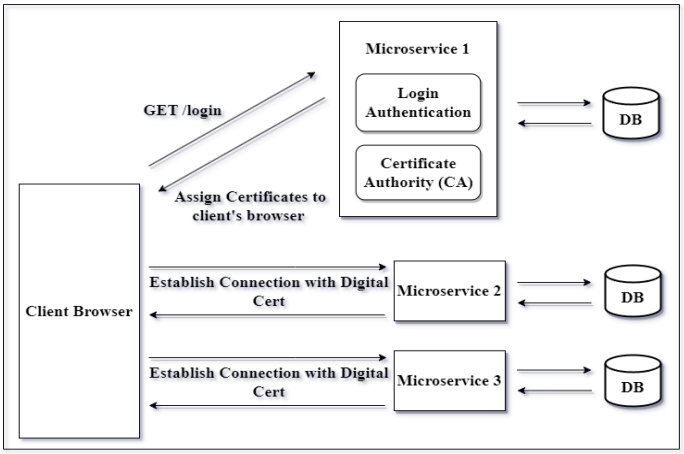

Fig. 3: An overview diagram on Mutual Transport Layer Security (mTLS) [16]

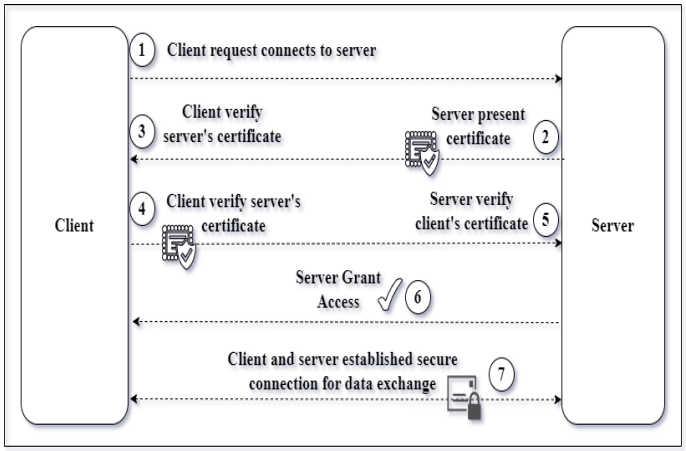

Fig. 4: Mutual Transport Layer Security (mTLS) Handshake Process [17]

has it capability to provides additional features in the gateway and provide single entry point authentication to secure client request in a centralized authentication server while token-based authentication has it own flexibility o n t he integration with different type of secure token which provides the concept ”one token for all” access to improve the authentication experience. Moreover, Mutual Transport Layer Security (mTLS) provides strong end-to-end encrypted authentication in a decentralized authentication mechanism which better supports message confidentiality a nd integrity.

One of the interesting findings i n b etween t hese t wo authentication mechanisms are the limitations that we discovered are likely the same. Both mechanism’s limitations were surrounded with its performance, security consideration and single point of failure. The most obvious finding t o emerge from the limitations is that a single point of failure in both mechanisms could bring up a huge impact to the distributed nature when the microservices-based system relies on one single entry point. Next, performance and security concerns described the increasing of latency during the authentication process and risk in data transport security during communication between microservices. Another important finding among these authentication mechanisms is that Mutual Transport Layer Security (mTLS) does not suffer from a single point of failure while communicating between each microservice. It provides strong end-to-end certificate authentication which no longer relies on any third party authentication server compared to API Gateway and token-based mechanism. Hence, it could conceivably be hypothesised that decentralized authentication mechanisms might be a better solution for microservices-based architecture.

Last but not least, these findings might help others to better understand the implementation of different types of authentication mechanisms in a microservices-based software architecture. Future research is required to address the following questions: Is Mutual Transport Layer Authentication (mTLS) a more suitable solution to the microservices-based architecture compared to API gateway and token-based authentication?

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:57:20 UTC from IEEE Xplore. Restrictions apply.

Is decentralized authentication greater than centralized authentication ? Will the API gateway [10] and token-based authentication face a single point of failure cause a scalability bottleneck? Performance evaluation on the authentication mechanisms. Therefore, future research and attestation efforts are required.

V. Conclusion

The main goal of the current study was to gain a better understanding of the authentication mechanism in microservices-based software architecture and determine the strengths and limitations in each mechanism. The present study lays the groundwork for future research into the implementation of authentication mechanisms in microservices-based architecture. We would like to suggest that decentralized authentication mechanisms such as mutual transport layer security (mTLS) should be highly focused compared to centralized authentication mechanisms due to the demands of dynamic digital landscape in the distributed nature. In terms of adaptability and scalability across distributed nodes, decentralized authentication claims that no single point of failure can disrupt the harmony of authentication. Therefore, We believe that the authentication mechanism is one of the greatest challenges that need to be addressed in the architecture. Moreover, future research should address the questions raised by this research.

Acknowledgment

Firstly, I wish to express my gratitude to Mr Ding Ying Hong for his constant support. Ultimately, I would like to thank my supervisors, Dr See Kwee Teck and Professor Lim Tong Ming who provided the help, guidance and support during my research study.

References

[1] K. Walsh and J. Manferdelli, "Mechanisms for mutual attested microservice communication," in UCC 2017 Companion - Companion Proceedings of the 10th International Conference on Utility and Cloud Computing, Association for Computing Machinery, Inc, Dec. 2017, pp. 59–64. doi: 10.1145/3147234.3148102.

[2] J. T. Zhao, S. Y. Jing, and L. Z. Jiang, "Management of API Gateway Based on Micro-service Architecture," in Journal of Physics: Conference Series, Institute of Physics Publishing, Oct. 2018. doi: 10.1088/1742-6596/1087/3/032032.

[3] T. Yarygina and A. H. Bagge, "Overcoming Security Challenges in Microservice Architectures," in Proceedings - 12th IEEE International Symposium on Service-Oriented System Engineering, SOSE 2018 and 9th International Workshop on Joint Cloud Computing, JCC 2018, Institute of Electrical and Electronics Engineers Inc., May 2018, pp. 11–20. doi: 10.1109/SOSE.2018.00011.

[4] A. Barabanov and D. Makrushin, "AUTHENTICATION AND AUTHORIZATION IN MICROSERVICE-BASED SYSTEMS: SURVEY OF ARCHITECTURE PATTERNS."

[5] M. O. Pahl and L. Donini, "Securing IoT microservices with certificates," in IEEE/IFIP Network Operations and Management Symposium: Cognitive Management in a Cyber World, NOMS 2018, Institute of Electrical and Electronics Engineers Inc., Jul. 2018, pp. 1–5. doi: 10.1109/NOMS.2018.8406189.

[6] X. He and X. Yang, "Authentication and Authorization of End User in Microservice Architecture," in Journal of Physics: Conference Series, Institute of Physics Publishing, Nov. 2017. doi: 10.1088/1742-6596/910/1/012060.

[7] Banati, A., Kail, E., Karoczkai, K., Kozlovszky, M. (2018). Authentication and authorization orchestrator for microservice-based software architectures. 2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics, MIPRO 2018 - Proceedings, 1180–1184. https://doi.org/10.23919/MIPRO.2018.8400214

[8] Henrique, W., Almeida, C., De Aguiar Monteiro, L., Hazin, R. R., Cavalcanti De Lima, A., Ferraz, F. S. (n.d.). Survey on Microservice Architecture-Security, Privacy and Standardization on Cloud Computing Environment.

[9] Liu, S., Zhang, Z. J., Cui, Y., Zhang, Y. (2021). An Industrial-Grade API Secure Access Gateway in the Cloud-Edge Integration Scenario. Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering, LNICST, 354, 57–69. https://doi.org/10.1007/978-3-030-69514-9_6

[10] https://doi.org/10.1007/978-3-030-69514-9_6 Lu, D., Huang, D., Walenstein, A., Medhi, D. (2017). A Secure Microservice Framework for IoT. Proceedings - 11th IEEE International Symposium on Service-Oriented System Engineering, SOSE 2017, 9–18. https://doi.org/10.1109/SOSE.2017.27

[11] Shulin, Y., Jieping, H. (2020). Research on Unified Authentication and Authorization in Microservice Architecture. International Conference on Communication Technology Proceedings, ICCT, 2020-October, 1169–1173. https://doi.org/10.1109/ICCT50939.2020.9295931

[12] Yang, J., Hou, H., Li, H., Zhu, Q. (2021). User Fast Authentication Method Based on Microservices. Proceedings of 2021 IEEE International Conference on Power Electronics, Computer Applications, ICPECA 2021, 93–98. https://doi.org/10.1109/ICPECA51329.2021.9362656

[13] A. Akbulut and H. G. Perros, "Software Versioning with Microservices through the API Gateway Design Pattern," in 2019 9th International Conference on Advanced Computer Information Technologies (ACIT), IEEE, Jun. 2019, pp. 289–292. doi: 10.1109/ACITT.2019.8779952.

[14] M. Ozkaya, "API gateway pattern," Medium, https://medium.com/design-microservices-architecture-with-patterns/api-gateway-pattern-8ed0ddfce9df (accessed Jun. 6, 2023).

[15] O. Updated: 02/14/2023 - 11:14 Time to read: 8 , "What is token-based authentication?," What Is Token-Based Authentication?, https://www.okta.com/identity-101/what-is-token-based-authentication/ (accessed Jun. 6, 2023).

[16] D. Warburton, "What is MTLS?," F5 Labs, https://www.f5.com/labs/learning-center/what-is-mtls (accessed Jun. 6, 2023).

[17] "What is MTLS? — mutual TLS — cloudflare," What is mutual TLS (mTLS)?, https://www.cloudflare.com/learning/access-management/what-is-mutual-tls/ (accessed Jun. 6, 2023).

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 13:57:20 UTC from IEEE Xplore. Restrictions apply.